

GreenStep Challenge Admin Console

Product Documentation

Team 2

Josh Dunlap Eli Goldberger Khue Vo Rudy Vergara

CISC 480: Senior Capstone, Section 01

University of St. Thomas

Community Partner: Minnesota Pollution Control Agency (MPCA)

May 22, 2026

Document Type: Product Documentation

Project: GreenStep Challenge Admin Console

Repository: https://github.com/CISC-480-GreenStep/sp26_team2_greenstepchallenge_admin

Live URL: <https://greenstep.auriga.fyi>

Version: v0.9.0

Submission Date: May 22, 2026

Contents

1	Product Requirement Document	3
1.1	Purpose	3
1.2	Target Users	3
1.3	Use Cases	3
1.4	User Stories	5
2	User Experience and Interface Design Specification	7
2.1	User Assumptions	7
2.2	Usage Constraints	7
2.3	User Story Map	7
2.4	User Journey	8
2.5	Accessibility and Inclusive Design Considerations	8
3	Technical Design Document	9
3.1	Architectural Overview	9
3.2	C4 – Level 1: System Context	9
3.3	C4 – Level 2: Containers	10
3.4	C4 – Level 3: Components inside the SPA	12
3.5	Data Design	12
3.6	Sequence Diagram: Login (US-1)	14
3.7	Sequence Diagram: Create Challenge (UC-1)	14
3.8	Class / Module Design (Data Layer)	16
3.9	Algorithm Explanation: Auto-Create Theme on Challenge Submit	16
3.10	Deployment Plan	17
4	Source Code Documentation	18
4.1	Code Structure	18
4.2	Naming Conventions	18
4.3	Commenting Standards	19
4.4	Style Enforcement	19
5	QA Activities Document	21
5.1	Test Strategy	21
5.2	Test Cases	21
5.3	Defect Reports	22
5.4	Known Limitations	23
	References	24

1 Product Requirement Document

1.1 Purpose

The GreenStep Challenge Admin Console is the back-office tool that staff at the Minnesota Pollution Control Agency (MPCA) use to design, launch, and measure sustainability challenges across their workforce. Before this tool, Kristin Mroz-Risse, the agency’s Sustainability Program Coordinator, ran each annual challenge as a sprawling network of spreadsheets, manual emails, and ad hoc reporting. The admin console replaces that workflow with a single web application backed by a managed Postgres database, role-based access control, and a customizable dashboard for monitoring engagement in real time.

1.2 Target Users

- **SuperAdmin** (Kristin and her direct delegates): full control over users, groups, challenges, templates, and deactivation. Can invite new admins.
- **Admin** (department-level coordinators): can create and manage challenges, groups, and templates; cannot create or deactivate top-level users.
- **GeneralUser** (the rest of MPCA staff): read-only inside the admin app; full participation happens in the sibling mobile application built by Team 1.

1.3 Use Cases

Use Case UC-1: Launch a sustainability challenge from a template

ID	UC-1
Name	Launch a sustainability challenge from a template
Primary actor	Admin
Stakeholders	MPCA staff (will participate); Kristin (program owner)
Preconditions	Admin is signed in; at least one challenge template exists; at least one group exists.
Trigger	Admin clicks <i>New Challenge</i> on the Challenges page.

Main flow

1. Admin opens the Challenges page and clicks **New Challenge**.
2. The Challenge Form renders with a template picker, field grid, and a live mobile preview on the right.
3. Admin selects an existing template (e.g. “Earth Month Challenge”). The form’s name, description, category, status, and color palette are pre-filled. The chosen template’s actions are staged for attachment after the challenge is created.
4. Admin adjusts the start date, end date, and target group, then tweaks the four challenge colors using the inline color pickers. The mobile preview updates in real time.
5. Admin clicks **Create Challenge**.
6. The system validates required fields, INSERTs the challenge row, attaches the staged actions via `challenge_actions`, and writes an `activity_logs` entry.
7. The system redirects to `/challenges` where the new challenge appears in the list.

Alternate flow A1	At step 3, Admin skips the template picker and fills the form from scratch. The remaining steps are unchanged.
Alternate flow A2	At step 5, validation fails (missing name or date). The form highlights the offending field and submission is blocked until corrected.
Exception E1	At step 6, Supabase rejects the INSERT because the caller does not satisfy the <code>admin_write</code> Row-Level Security policy. The form surfaces “Permission denied” and the challenge is not created.
Postconditions	A new row exists in <code>challenges</code> ; N rows exist in <code>challenge_actions</code> ; one new row exists in <code>activity_logs</code> ; the participantCount column starts at zero.

Use Case UC-2: Triage engagement mid-challenge

ID	UC-2
Name	Triage engagement mid-challenge
Primary actor	Admin or SuperAdmin
Stakeholders	Kristin (program owner); challenge participants who may need a nudge
Preconditions	At least one challenge is in Active status with participants and logged actions.
Trigger	Admin opens the dashboard during a running challenge.

Main flow

1. Admin lands on the dashboard. The default layout renders the Active Challenges, Top Participants, Recent Activity, and Participation Breakdown widgets.
2. Admin filters the dashboard to the running challenge using the per-page challenge filter chip. All widgets refresh.
3. Admin scans the Top Participants widget and notes the long tail of users with zero or one logged actions.
4. Admin opens the dashboard’s Customize panel and applies the *Analytics Deep Dive* layout preset to expose the Participation Report widget.
5. Admin filters the Participation Report by date range (e.g. the last 7 days) and exports a CSV.
6. Admin opens the CSV in Excel, identifies users with no activity, and follows up by email outside the system.

Alternate flow A1	At step 4, Admin instead drags an existing widget to resize it; the new layout is persisted to <code>localStorage</code> and survives a page refresh.
Exception E1	At step 5, if no rows match the filter, the table renders an empty state (“No participation in this range”) and the Export button is disabled.
Postconditions	A CSV download has been triggered; no database state is modified; the dashboard layout for this user is updated in <code>localStorage</code> .

1.4 User Stories

The following four user stories were prioritized for the spring 2026 release. Each carries explicit acceptance criteria.

US-1 – Passwordless sign-in

As an MPCA staff member, I want to sign in to the admin console using a one-time code emailed to me so that I don’t have to manage yet another password.

Acceptance criteria:

- Entering an unregistered email shows “Email not recognized” without sending a code.
- Entering a registered **Active** email triggers an 8-digit code delivered via Supabase Auth within 30 seconds.
- Entering a registered **Deactivated** email shows “Account deactivated” and does *not* send a code.
- Submitting the correct code within its TTL completes sign-in and lands the user on the dashboard.
- Submitting an incorrect code shows an inline error and leaves the form populated.

US-2 – Create a custom group

As an admin, I want to create a group (department, cohort, or city) with a name and description so that I can scope a challenge to a specific population.

Acceptance criteria:

- Only Admin and SuperAdmin roles see the “New Group” button.
- Group name is required; description is optional.
- The new group appears immediately in the Groups list with member count = 0 and challenge count = 0.
- The group can be selected as the target of a new challenge without a page refresh.

US-3 – Customize the dashboard

As an admin, I want to rearrange and resize dashboard widgets so that the metrics I check daily are visible without scrolling.

Acceptance criteria:

- Clicking “Customize” enters edit mode; widgets gain a grip handle and resize corners.
- Widgets can be dragged to any 12-column grid cell and resized from any corner.
- “Save” persists the layout to `localStorage` for the current user.
- “Cancel” restores the layout that was active when edit mode was entered.
- Applying a Quick Layout Preset (Default, Executive Summary, Analytics Deep Dive, Compact Overview) overwrites the visible layout but does not save until the user clicks Save.
- The layout survives a hard page refresh.

US-4 – Export a participation report

As an admin, I want to export a filtered participation report to CSV so that I can share it with leadership in their preferred format.

Acceptance criteria:

- The report can be filtered by challenge and by a date range.
- The export button is disabled when zero rows match the current filter.
- The exported CSV contains the visible columns: User, Challenge, Action, Category, Date, Notes.
- Filenames follow the pattern `participation-YYYY-MM-DD.csv`.
- User and Challenge cells in the on-screen table are clickable and navigate to the corresponding detail page.

2 User Experience and Interface Design Specification

2.1 User Assumptions

- Users are MPCA staff with intermediate computer literacy; comfortable with web applications such as Outlook, SharePoint, and Concur.
- Users access the admin console primarily from a corporate laptop on the MPCA network; secondary access from personal devices is supported but not the primary use case.
- Users are familiar with the GreenStep program’s vocabulary (“Action,” “Challenge,” “Category”). Onboarding new admins outside MPCA may require glossary support that lives outside this product.
- English is the only supported language for the initial release.
- Users have a corporate email account that can receive the OTP code in under one minute.

2.2 Usage Constraints

- **Network:** The application requires a live internet connection; there is no offline mode. The static SPA bundle is served from Vercel’s global edge network and all data round-trips go to Supabase over HTTPS.
- **Browser:** Tested on the current versions of Chrome, Edge, Safari, and Firefox. The application targets modern evergreen browsers; IE 11 is explicitly not supported.
- **Email scanners:** MPCA staff inboxes are protected by corporate link-scanning that pre-fetches one-time URLs. For that reason the auth flow uses an 8-digit OTP code rather than a magic link.
- **Data residency:** All data lives in Supabase’s US-East-1 region.
- **Mobile:** The admin app is responsive but optimized for desktop. Field-level data entry (e.g. logging a completed action) happens in the sibling mobile app, not here.

2.3 User Story Map

Figure 1 shows the admin’s mental model of the product, with the backbone (left to right) tracing the typical program cycle and stories grouped into MVP, polish, and future releases.

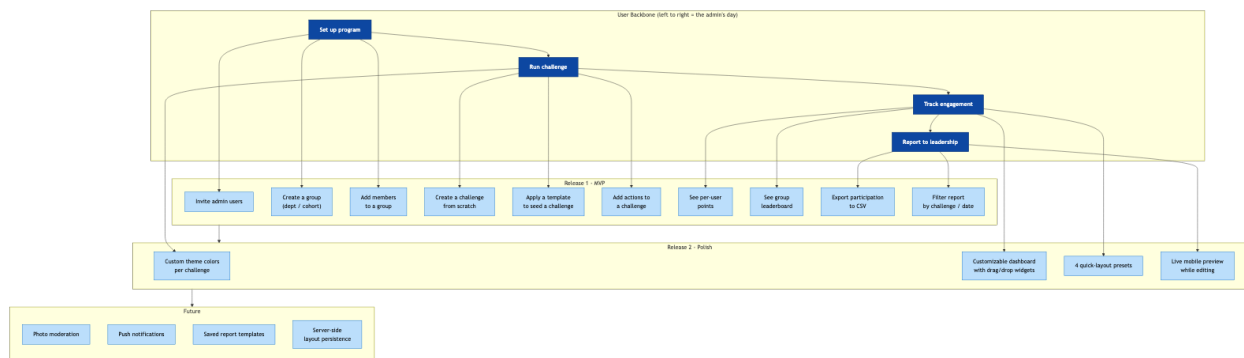


Figure 1: User story map. The horizontal backbone follows the admin’s program-management workflow; vertical slices are prioritized into Release 1 (MVP, shipped), Release 2 (polish, shipped), and Future (handed off).

2.4 User Journey

Figure 2 traces a representative admin journey across the lifecycle of a single multi-week challenge, showing where the product currently delights (mobile preview, dashboard filtering, CSV export) and where friction remains (manual follow-up to dormant users).

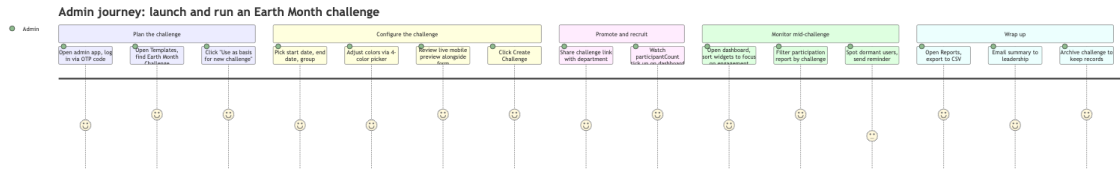


Figure 2: Admin user journey: launching and running an Earth Month challenge. Higher numbers indicate higher admin satisfaction at that step.

2.5 Accessibility and Inclusive Design Considerations

- Color contrast.** The MUI v7 dark theme in `src/lib/theme.js` was chosen partly for accessibility: brand greens are layered on dark surfaces with contrast ratios at or above WCAG AA for body text. Challenge color pickers default to the seeded MPCA palette, but the admin is free to choose any hex color, including low-contrast combinations – this is a documented trade-off in favor of creative freedom.
- Keyboard navigation.** All interactive elements are rendered through MUI components which carry native keyboard affordances (tab order, enter/space activation, focus rings). The dashboard’s drag-and-drop grid is the one exception; keyboard-only users cannot currently reorder widgets and must rely on the Quick Layout Presets instead.
- Screen readers.** Form fields are labeled (MUI `TextField` `label`) so they announce correctly. Custom clickable counts on list pages use `<button>` rather than `<div onClick>` so they are reachable via assistive tech.
- Cognitive load.** The Customize Dashboard panel applies several HCI principles: affordance cues (grip handles, dashed borders in edit mode), feedback (snackbar confirmations after save), error prevention (cannot remove every widget; cancel/reset always available), and progressive disclosure (edit controls only appear once edit mode is engaged).
- Language.** The product is English-only. The Action and Category vocabulary was adapted directly from MPCA’s existing GreenStep program so the language matches what staff already use.

3 Technical Design Document

3.1 Architectural Overview

The system is a single-page React 19 application hosted on Vercel that talks directly to a managed Supabase project. Authorization is enforced not in the React code but inside Postgres via Row-Level Security (RLS) policies that read the role claim from the caller's JWT. Two narrow Vercel serverless functions exist for privileged operations that cannot be performed with the anon key (inviting a user, hard-flipping the active status of a user account).

3.2 C4 – Level 1: System Context

Figure 3 shows the admin console in its environment. The two human actors are MPCA staff (using this admin app) and challenge participants (using the sibling mobile app built by Team 1). Both apps share a single Supabase database.

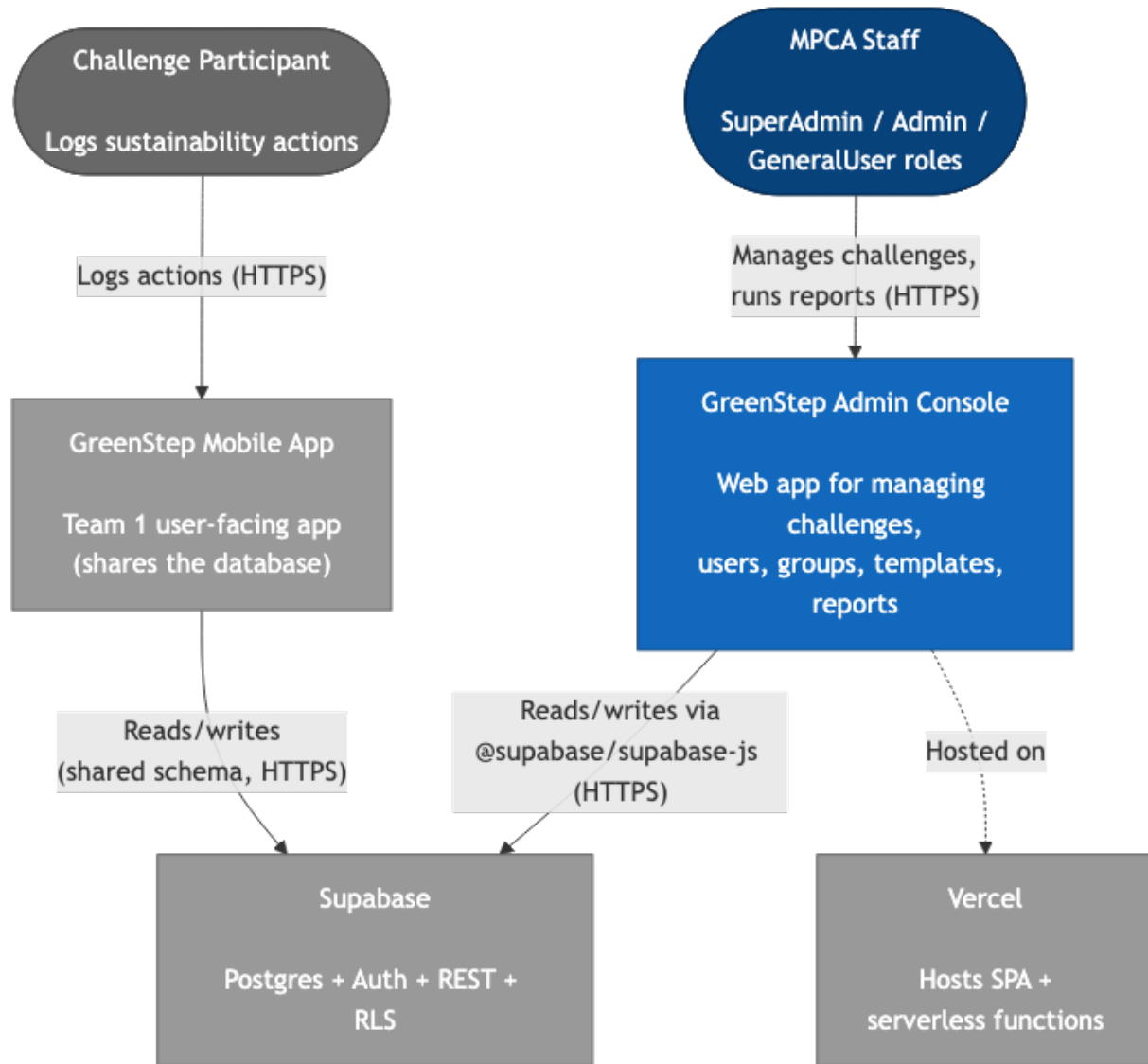


Figure 3: C4 Level 1 – System context diagram.

3.3 C4 – Level 2: Containers

Figure 4 shows the deployable units inside the admin console's boundary. The crucial design decision is that the React SPA talks to Supabase directly with the anon key; authorization lives in RLS policies, not in a middle-tier API.

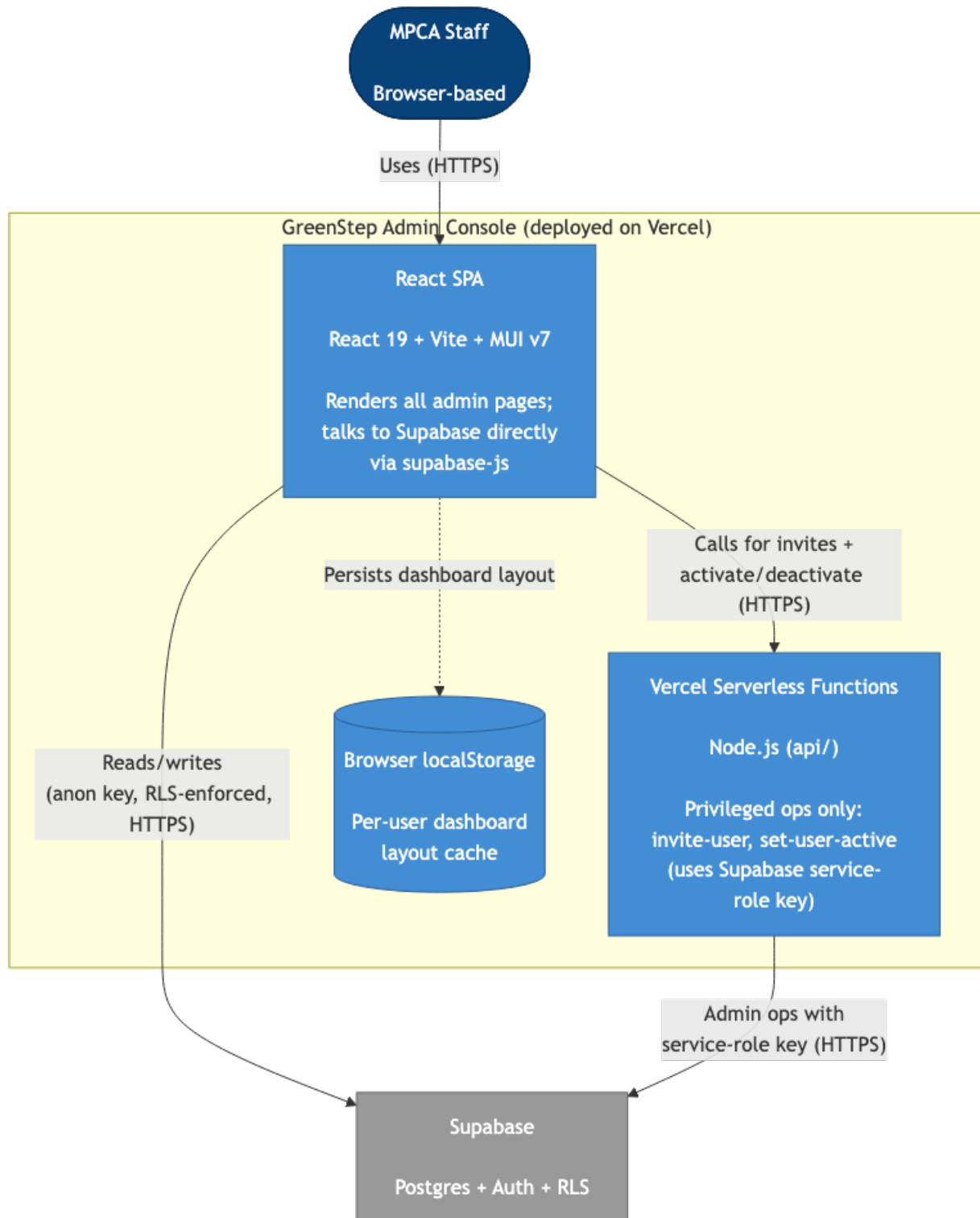


Figure 4: C4 Level 2 – Container diagram. The SPA bypasses the need for a custom backend by relying on RLS to enforce authorization at the database layer.

3.4 C4 – Level 3: Components inside the SPA

Figure 5 shows how the SPA is organized internally, mirroring the source tree under `src/admin-app/src/`.

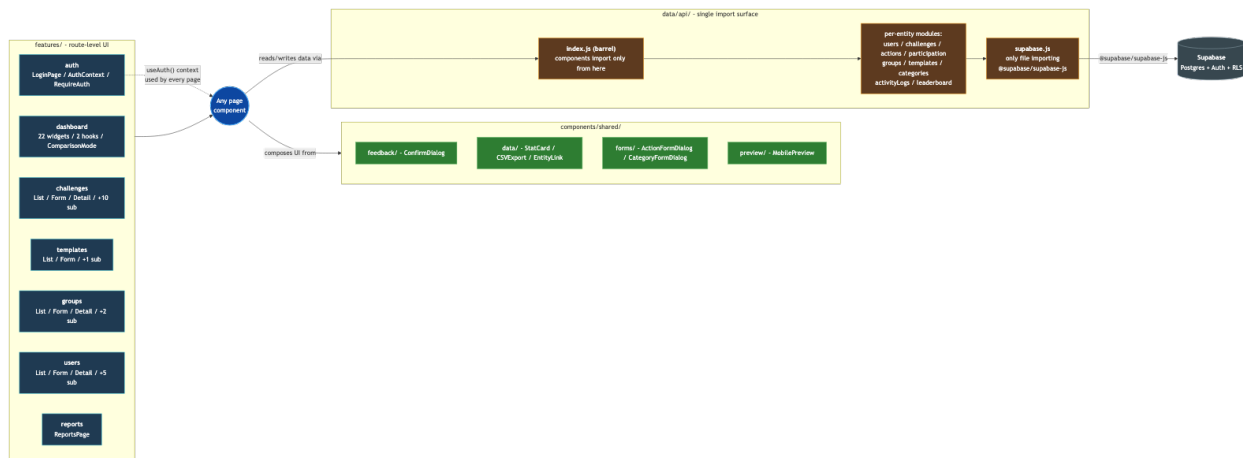


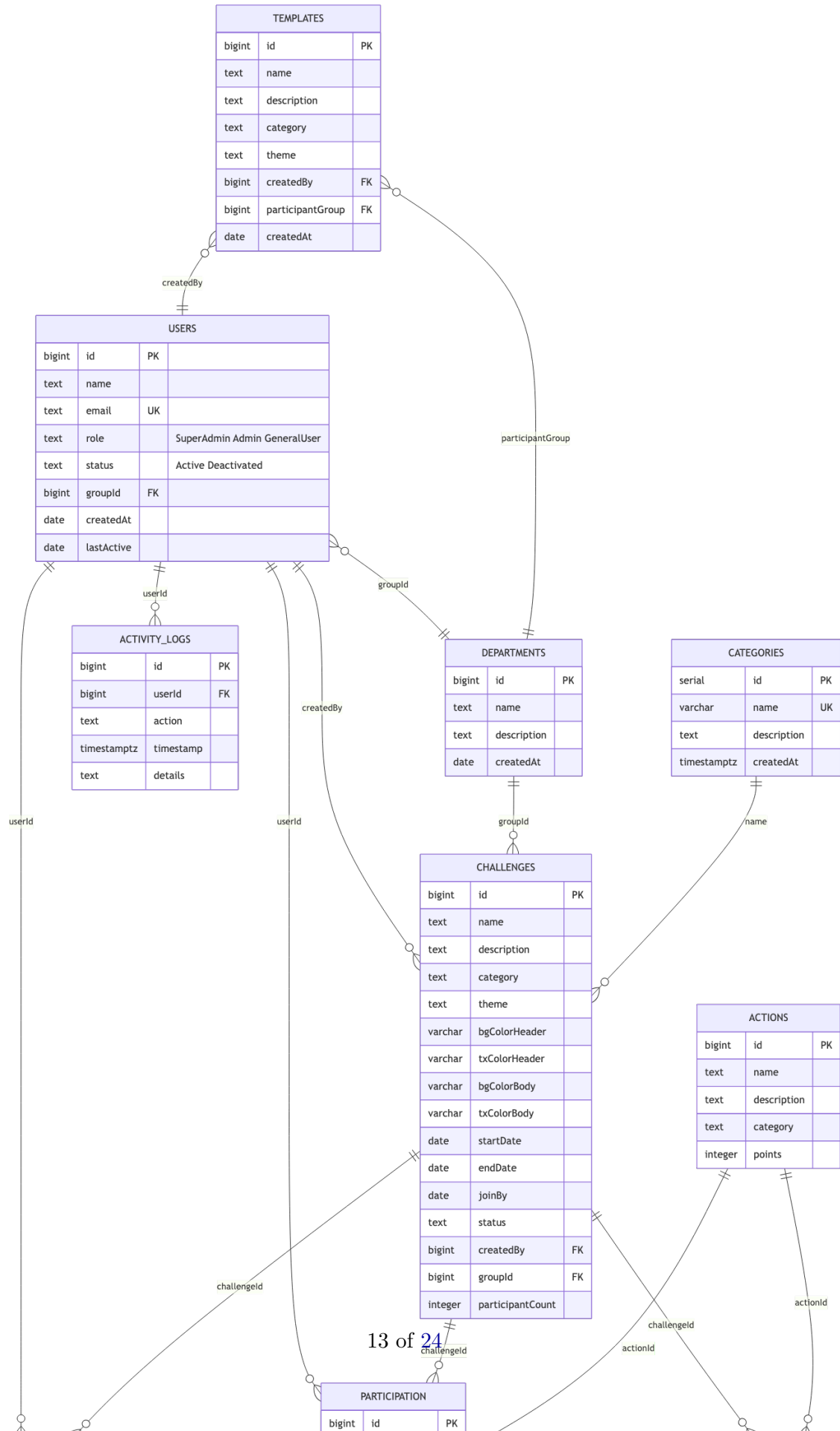
Figure 5: C4 Level 3 – SPA component view. Page components compose UI from `components/shared` and read/write data through the `data/api` barrel; only one file (`data/supabase.js`) imports the Supabase SDK.

Verified architectural invariants

1. Only `data/supabase.js` imports `@supabase/supabase-js`. (Grep: 1 match.)
2. No component bypasses the `data/api` barrel to import a sub-module directly. (Grep: 0 matches.)
3. Cross-feature components live in `components/shared/`. No feature folder imports another feature's components.
4. `lib/` stays small: 3 files (`constants.js`, `permissions.js`, `theme.js`).

3.5 Data Design

Figure 6 shows the entity-relationship diagram for the deployed Postgres schema. The schema is consolidated into the single migration file `supabase/migrations/001_initial_schema.sql`.



3.6 Sequence Diagram: Login (US-1)

Figure 7 shows the OTP-code login flow. The pre-flight `auth_email_is_registered` RPC is a **security definer** function that runs without RLS so the anon-keyed client can ask “does this email exist and is the account active?” without being able to read the `users` table directly.

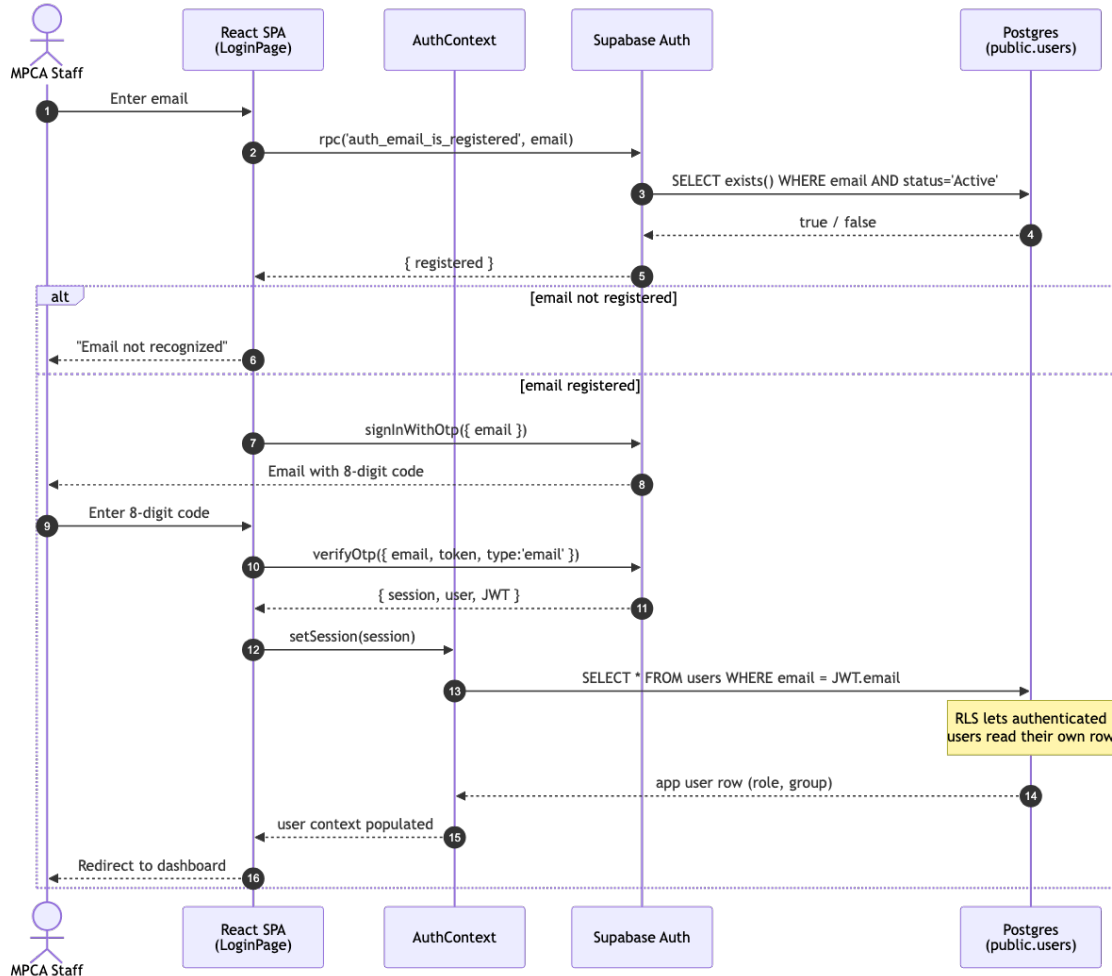


Figure 7: Sequence diagram – 8-digit OTP login flow.

3.7 Sequence Diagram: Create Challenge (UC-1)

Figure 8 traces the create-challenge flow, including the three parallel pre-flight reads (categories, groups, templates) and the RLS check that gates the INSERT.

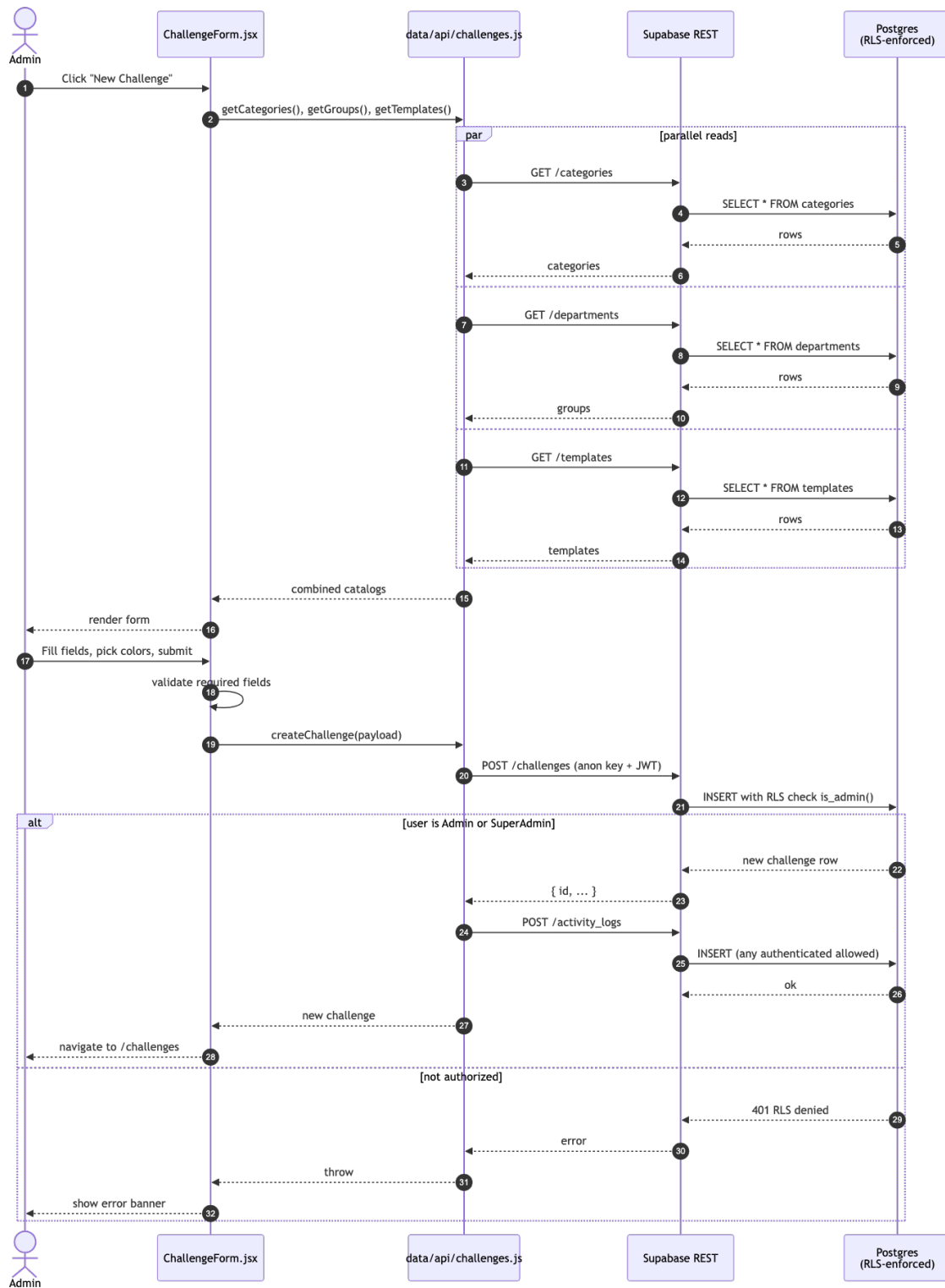


Figure 8: Sequence diagram – creating a new challenge. The `admin.write` RLS policy on the `challenges` table denies the `INSERT` for callers whose JWT role is not Admin or SuperAdmin.

3.8 Class / Module Design (Data Layer)

Figure 9 shows the structure of the `data/api` module. The `index.js` barrel is the sole public surface; components are forbidden by ESLint rule from reaching into sub-modules.

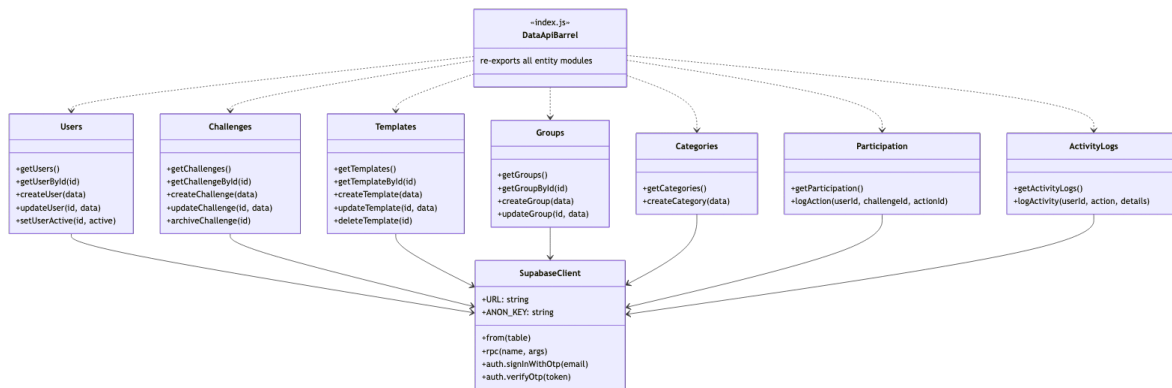


Figure 9: Data-layer module structure. Each per-entity module owns one Postgres table’s CRUD operations.

3.9 Algorithm Explanation: Auto-Create Theme on Challenge Submit

When an admin clicks **Create Challenge**, the form must reconcile its UI – which exposes free-form color pickers – with the schema, which stores colors inline on **challenges**. The algorithm in pseudocode:

```

def build_challenge_payload(form, global_categories, user):
    # 1. Resolve the chosen category name to its FK id.
    category = find(global_categories, name == form.category)
    if not category:
        raise UserVisibleError("Please select a category.")

    # 2. Compose the challenge row, including the four split
    #    color columns.
    return {
        "name": form.name,
        "description": form.description,
        "categoryId": category.id,
        "groupId": form.groupId or None,
        "startDate": form.startDate,
        "endDate": form.endDate,
        "status": form.status,
        "bgColorHeader": form.bgColorHeader,
        "txColorHeader": form.txColorHeader,
        "bgColorBody": form.bgColorBody,
        "txColorBody": form.txColorBody,
        "createdBy": user.id,
    }
  
```

Listing 1: Algorithm: build challenge payload for INSERT

3.10 Deployment Plan

Figure 10 shows the production deployment topology.

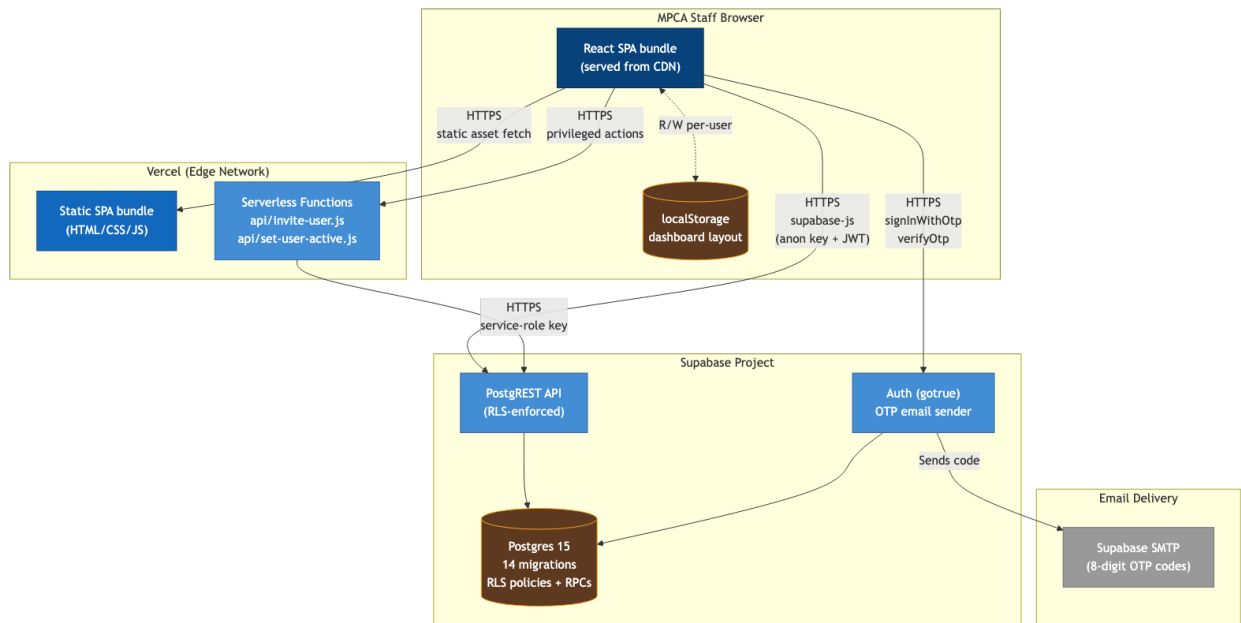


Figure 10: Deployment diagram. The SPA is served from Vercel’s edge network; data and auth go directly to Supabase. Two serverless functions handle privileged operations using Supabase’s service-role key.

Continuous deployment

- Push to `main` triggers a Vercel production deploy. Pull requests trigger preview deploys with their own URL.
- Database changes are version-controlled in `supabase/migrations/` and applied via `supabase db push`. The CLI reconciles the local file list against the `supabase_migrations.schema_migrations` table on the live database.
- Environment variables (`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`) are set in the Vercel project; the service-role key lives only in the serverless-function environment and never reaches the client bundle.

4 Source Code Documentation

4.1 Code Structure

The application source lives under `src/admin-app/src/` and is organized by feature, not by file type. Every page is a feature folder; cross-cutting components live in `components/shared/`; all data access goes through `data/api/`.

```
src/admin-app/src/
+- main.jsx           React entry
+- App.jsx           Route table
+- lib/
| +- theme.js        MUI dark theme
| +- permissions.js  can(role, perm)
| +- constants.js     Palettes, status maps
+- components/
| +- layout/         AdminLayout, Sidebar, TopBar
| +- shared/
|   +- feedback/     ConfirmDialog
|   +- data/          StatCard, CSVExport, EntityLink
|   +- forms/         ActionFormDialog, CategoryFormDialog
|   +- preview/       MobilePreview
+- data/
| +- supabase.js      sole importer of @supabase/supabase-js
| +- api/
|   +- index.js       barrel; the only file features import
|   +- users.js, challenges.js, templates.js, groups.js, ...
+- features/
  +- auth/           LoginPage, AuthContext, RequireAuth
  +- dashboard/      22 widgets, ComparisonMode
  +- challenges/     List, Form, Detail, +10 sub-components
  +- templates/     List, Form, +1 sub
  +- groups/        List, Form, Detail, +2 sub
  +- users/         List, Form, Detail, +5 sub
  +- reports/       ReportsPage
```

Listing 2: Source tree (abbreviated)

4.2 Naming Conventions

- **Files.** React components use `PascalCase.jsx` (e.g. `ChallengeForm.jsx`). Hooks use `useCamelCase.js`. Non-component modules use `camelCase.js` (e.g. `aggregations.js`).
- **Folders.** Feature folders are plural nouns (`challenges/`, `users/`). Component sub-folders are singular nouns (`components/shared/feedback/`).
- **Variables and functions.** `camelCase`. React event handlers prefixed with `handle` (`handleSubmit`, `handleApplyTemplate`). Props that are callbacks prefixed with `on` (`onChange`, `onSelect`).
- **Constants.** `UPPER_SNAKE_CASE` (`CHALLENGE_STATUSES`, `ROLES`).
- **Database columns.** Postgres identifiers use `camelCase` (quoted, e.g. `"groupId"`). Table names are plural `snake_case` (`challenge_participants`). Foreign keys end in `Id` (`userId`, `challengeId`).

4.3 Commenting Standards

File-header docblock

Every source file opens with a JSDoc block giving a one-line `@summary` and any context the file's purpose can't convey on its own:

```
/**
 * @file ChallengeForm.jsx
 * @summary ChallengeForm -- create or edit a challenge.
 *
 * Renders three sub-views, all under one route:
 *   - TemplatePicker: only on create, lets the user pre-fill from a
 *     template
 *   - ChallengeFieldsSection: the field grid
 *   - ActionsEditor: only on edit, manages the challenge's actions
 *
 * This component owns the form state and the submit pipeline; the
 * sub-components are pure presentational.
 */
```

Listing 3: Example file-header docblock

JSDoc on exported functions

Every exported function carries a JSDoc block declaring its parameters and return shape. Generated documentation is not shipped; the value is in-IDE hover and onboarding.

Inline comments explain *why*, not *what*

Inline comments justify non-obvious decisions; they never restate what the code already says.

```
// security definer + explicit search_path so the inner SELECT
// bypasses RLS on public.users (otherwise the admin_write policy
// would recursively call this function while evaluating itself).
create or replace function public.current_user_role() ...
```

Listing 4: Example: a *why* comment

TODO and FIXME format

TODO(name): short description -- issue#NN or FIXME(name): Every TODO points to a tracked GitHub issue so the next developer can find the context.

4.4 Style Enforcement

- **Prettier** owns whitespace, quote style, semicolons, and line length. Configured via `.prettierrc.json`; runs in CI and on `npm run format`.
- **ESLint flat config** enforces import order, no-unused, max-lines, and the React recommended set. Custom rule: no file in `src/admin-app/src/` may exceed 300 lines of code; offenders fail the lint job.

- **File-size policy.** Components over 200 lines are candidates for decomposition; over 300 lines is a hard cap. See the version-history entries for v0.7.0 and v0.7.1 for the project-wide decomposition that brought every file under the cap.

5 QA Activities Document

5.1 Test Strategy

The team adopted a pragmatic, manual-first test strategy appropriate to a one-semester capstone:

- **Static analysis** on every commit: ESLint catches unused imports, React Hooks rule violations, and the 300-line cap; Prettier catches formatting drift.
- **Build verification** on every pull request: Vercel builds a preview deploy from the PR's HEAD. A red build is a blocking merge condition.
- **Manual exploratory testing** on the preview URL before merging each PR. Each PR description carries an explicit “Test plan” checklist (see for example PRs #81, #83, #85).
- **Schema reconciliation** on every migration: `supabase db push --dry-run` is run against the live project to confirm only intended migrations would be applied; `supabase migration list --linked` is diffed against the local file list to confirm no drift.

5.2 Test Cases

The table below documents the exploratory-test pass that verified each user story and major use case for the final release. *Expected* is the acceptance criterion; *Actual* is the observed behavior on the production deploy.

ID	Scenario	Expected	Actual	P/F
TC-1	Login: unregistered email	“Email not recognized” shown; no code sent.	Matches expected.	P
TC-2	Login: deactivated user	“Account deactivated” shown; no code sent.	Matches expected.	P
TC-3	Login: valid email, correct code	User is signed in and lands on dashboard within 2s.	Matches expected.	P
TC-4	Login: valid email, wrong code	Inline error; form remains populated.	Matches expected.	P
TC-5	Create group (Admin)	Group appears immediately in list with 0/0 counts.	Matches expected.	P
TC-6	Create group (GeneralUser)	“New Group” button is not rendered.	Button hidden; direct POST attempt is denied by RLS.	P
TC-7	Create challenge from template	Form pre-fills; actions stage; redirect to /challenges after save.	Matches expected.	P
TC-8	Create challenge: missing required field	Browser-native validation blocks submit.	Matches expected.	P
TC-9	Dashboard: drag-and-drop reorder	New position persists across hard refresh.	Matches expected.	P
TC-10	Dashboard: apply preset	Visible layout changes; Save persists; Cancel restores.	Matches expected.	P
TC-11	Reports: filter by challenge + date	Row count updates; CSV exports filtered rows.	Matches expected.	P

TC-12	Reports: empty filter	Empty state shown; Export disabled.	Matches expected.	P
TC-13	User detail: change group (Admin)	Group updates immediately; audit log row written.	Matches expected.	P
TC-14	User detail: deactivate (SuperAdmin)	User flips to Deactivated; cannot log in.	Matches expected.	P
TC-15	Mobile preview updates live	Typing in form fields updates the preview.	Matches expected.	P

5.3 Defect Reports

The following defects were filed, fixed, and verified during the spring 2026 release cycle. Severity uses a four-tier scale (Critical / High / Medium / Low). All listed defects are closed.

DR-1 – Create Challenge button silently fails (#82)

- **Severity:** Critical – core user story (US-2, UC-1) blocked.
- **Reproduce:** On the live deploy, click **New Challenge**, fill all required fields, click **Create Challenge**.
- **Expected:** Challenge is created; redirect to `/challenges`.
- **Actual:** Browser console showed `POST /rest/v1/challenges?select=* 400 (Bad Request)` with PostgreSQL error “Could not find the 'bgColorBody' column of 'challenges' in the schema cache”; no user-visible feedback.
- **Root cause:** The form had always written the four split-color columns (`bgColorHeader/txColorHeader/bgC`) but the deployed schema only carried the legacy single `theme text` column. The repository's `001_initial_schema.sql` described a different normalized schema that was never deployed, misleading the initial fix attempt.
- **Fix:** Added migration `014_challenge_split_colors.sql` adding the four columns, applied via `supabase db push`. Merged in PR #83. Subsequently issued PR #85 to reconcile the in-repo migration files with the actually-deployed schema and squash 14 migrations to 2 for handoff readability.
- **Status:** Closed; verified on production by creating a challenge end-to-end.

DR-2 – Dev login race condition

- **Severity:** High – blocked dev productivity for the whole team after the RLS migration.
- **Reproduce:** Click “Quick Login as Kristin”; observe that the dashboard renders before `user.role` is populated, causing role-gated widgets to flicker.
- **Expected:** Dashboard does not render until the app user row is loaded.
- **Actual:** Caller's `navigate("/")` fired before `RequireAuth` could see the new user.
- **Fix:** Load the app user inline inside `devLogin` before returning success. Merged as commit `ef9f3c5`.
- **Status:** Closed; verified by repeated Quick Login attempts with no flicker.

DR-3 – Magic-link tokens consumed by corporate email scanners

- **Severity:** High – blocked real users at MPCA even though local dev was unaffected.
- **Reproduce:** Send a magic-link to an `@mpca.mn.gov` account; click the link in the email.
- **Expected:** Successful sign-in.

- **Actual:** Sign-in fails with “token already consumed.” Microsoft Defender Safe Links pre-fetches the URL on email arrival, burning the one-time token before the user can click it.
- **Fix:** Migrated the auth flow from magic links to an 8-digit OTP code in PR #61. Code is delivered in plain text in the email body, which scanners do not pre-fetch.
- **Status:** Closed; verified on a live MPCA email account.

DR-4 – Cross-feature component imports

- **Severity:** Low – code-quality issue, no user-visible effect.
- **Reproduce:** `grep -r "features/challenges" src/admin-app/src/features/`.
- **Expected:** No feature folder imports another feature’s components.
- **Actual:** `features/templates/TemplateForm.jsx` imported `ActionFormDialog` and `CategoryFormDialog` from `features/challenges/components/`.
- **Fix:** Moved both dialogs into `components/shared/forms/` and updated import sites. Merged as PR #81 (closes #70).
- **Status:** Closed; the architectural invariant is now enforced by convention and verified by `grep`.

5.4 Known Limitations

The following are documented limitations rather than defects; they are out of scope for the spring 2026 capstone and are flagged for the next student team.

- No integration with the Team 1 mobile app beyond sharing the database schema.
- Dashboard layout persists to `localStorage` only; not synced across devices.
- `presets` and `preset_actions` legacy tables still exist on the live database even though the application no longer reads them; safe to drop in a future migration.
- The legacy `challenges.theme` single-color column is still read by some dashboard widgets while the form writes only the four split-color columns; widget code should migrate to read the split columns in a follow-up.

References

- [1] S. Brown, *The C4 model for visualising software architecture*. [Online]. Available: <https://c4model.com/>. [Accessed: May 21, 2026].
- [2] Supabase, “Row Level Security.” [Online]. Available: <https://supabase.com/docs/guides/auth/row-level-security>. [Accessed: May 21, 2026].
- [3] Supabase, “Database Migrations.” [Online]. Available: <https://supabase.com/docs/guides/deployment/database-migrations>. [Accessed: May 21, 2026].
- [4] Material UI, “Material UI v7 Documentation.” [Online]. Available: <https://mui.com/material-ui/>. [Accessed: May 21, 2026].
- [5] Vercel, “Functions Overview.” [Online]. Available: <https://vercel.com/docs/functions>. [Accessed: May 21, 2026].
- [6] J. Patton, *User Story Mapping: Discover the Whole Story, Build the Right Product*. O’Reilly Media, 2014.
- [7] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1.” [Online]. Available: <https://www.w3.org/TR/WCAG21/>. [Accessed: May 21, 2026].
- [8] Mermaid, “Mermaid Documentation.” [Online]. Available: <https://mermaid.js.org/>. [Accessed: May 21, 2026].